

Chapter 6. Processing Events with Stateful Functions

Imperative styles of programming are some of the oldest of all, and their popularity persists for good reason. Procedures execute sequentially, spelling out a story on the page and altering the program's state as they do so.

As mainstream applications became distributed in the 1980s and 1990s, the same mindset was applied to this distributed domain. Approaches like Corba and EJB (Enterprise JavaBeans) raised the level of abstraction, making distributed programming more accessible. History has not always judged these so well. EJB, while touted as a panacea of its time, fell quickly by the wayside as systems creaked with the pains of tight coupling and the misguided notion that the network was something that should be abstracted away from the programmer.

In fairness, things have improved since then, with popular technologies like [gRPC](#) and [Finagle](#) adding elements of asynchronicity to the request-driven style. But the application of this mindset to the design of distributed systems isn't necessarily the most productive or resilient route to take. Two styles of programming that better suit distributed design, particularly in a services context, are the dataflow and functional styles.

You will have come across dataflow programming if you've used [utilities like Sed or languages like Awk](#). These are used primarily for text processing; for example, a stream of lines might be pushed through a regex, one line at a time, with the output piped to the next command, chaining through stdin and stdout. This style of program is more like an assembly line, with each worker doing a specific task, as the products make their way along a conveyor belt. Since each worker is concerned only with the availability of data inputs, there have no "hidden state" to track. This is very similar to the way streaming systems work. Events accumulate in a stream processor waiting for a condition to be met, say, a join operation between two different streams. When the correct events are present, the join operation completes and the pipeline continues to the next com-

mand. So Kafka provides the equivalent of a pipe in Unix shell, and stream processors provide the chained functions.

There is a similarly useful analogy with functional programming. As with the dataflow style, state is not mutated in place, but rather evolves from function to function, and this matches closely with the way stream processors operate. So most of the benefits of both functional and dataflow languages also apply to streaming systems. These can be broadly summarized as:

- Streaming has an inherent ability for parallelization.
- Streaming naturally lends itself to creating cached datasets and keeping them up to date. This makes it well suited to systems where data and code are separated by the network, notably data processing and GUIs.
- Streaming systems are more resilient than traditional approaches, as high availability is built into the runtime and programs execute in a lossless manner (see the discussion of Event Sourcing in [Chapter 7](#)).
- Streaming functions are typically easier to reason about than regular programs. Pure functions are free from side effects. Stateful functions are not, but do avoid shared mutable state.
- Streaming systems embrace a polyglot culture, be it via different programming languages or different datastores.
- Programs are written at a higher level of abstraction, making them more comprehensible.

But streaming approaches also inherit some of the downsides. Purely functional languages must negotiate an impedance mismatch when interacting with more procedural or stateful elements like filesystems or the network. In a similar vein, streaming systems must often translate to the request-response style of REST or RPCs and back again. This has led some implementers to build systems around a functional core, which processes events asynchronously, wrapped in an *imperative shell*, used to marshal to and from outward-facing request-response interfaces. The “functional core, imperative shell” pattern keeps the key elements of the system both flexible and scalable, encouraging services to avoid side effects and express their business logic as simple functions chained together through the log.

In the next section we'll look more closely at why statefulness, in the context of stream processing, matters.

Making Services Stateful

There is a well-held mantra that statelessness is good, and for good reason. Stateless services start instantly (no data load required) and can be scaled out linearly, cookie-cutter-style.

Web servers are a good example: to increase their capacity for generating dynamic content, we can scale a web tier horizontally, simply by adding new servers. So why would we want anything else? The rub is that most applications aren't really stateless. A web server needs to know what pages to render, what sessions are active, and more. It solves these problems by keeping the state in a database. So the database is stateful and the web server is stateless. The state problem has just been pushed down a layer. But as traffic to the website increases, it usually leads programmers to cache state locally, and local caching leads to cache invalidation strategies, and a spiral of coherence issues typically ensues.

Streaming platforms approach this problem of where state should live in a slightly different way. First, recall that events are also facts, converging toward the stream processor like conveyor belts on an assembly line. So, for many use cases, the events that trigger a process into action contain *all the data the program needs*, much like the dataflow programs just discussed. If you're validating the contents of an order, all you need is its event stream.

Sometimes this style of stateless processing happens naturally; other times implementers deliberately enrich events in advance, to ensure they have all the data they need for the job at hand. But enrichments inevitably mean looking things up, usually in a database.

Stateful stream processing engines, like Kafka's Streams API, go a step further: they ensure all the data a computation needs is loaded into the API ahead of time, be it events or any tables needed to do lookups or enrichments. In many cases this makes the API, and hence the application, stateful, and if it were restarted for some reason it would need to reacquire that state before it could proceed.

This should seem a bit counterintuitive. Why would you want to make a service stateful? Another way to look at this is as an advanced form of caching that better suits data-intensive workloads. To make this clearer, let's look at three examples—one that uses database lookups, one that is event-driven but stateless, and one that is event-driven but stateful.

The Event-Driven Approach

Say we have an email service that listens to an event stream of orders and then sends confirmation emails to users once they complete a purchase. This requires information about both the order as well as the associated payment. Such an email service might be created in a number of different ways. Let's start by assuming it's a simple event-driven service (i.e., no use of a streaming API, as in [Figure 6-1](#)). It might react to order events, then look up the corresponding payment. Or it might do the reverse: reacting to payments, then looking up the corresponding order. Let's assume the former.

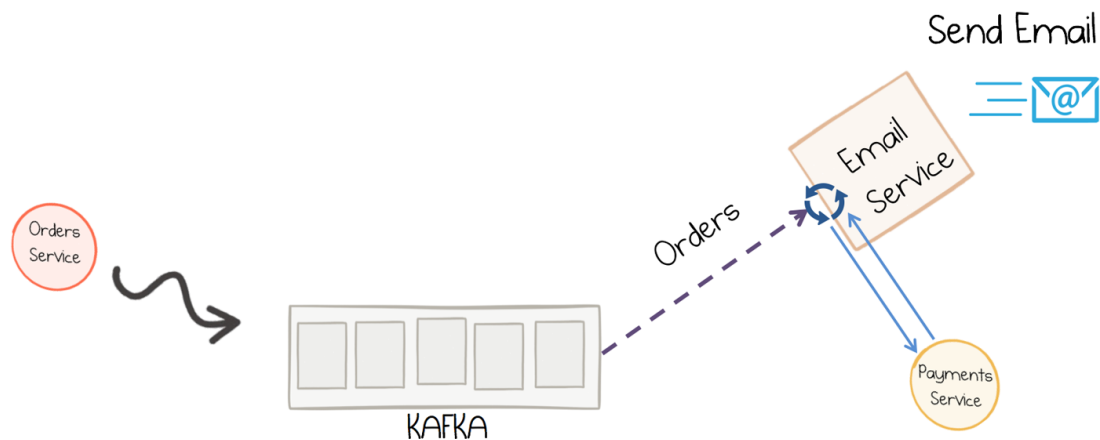


Figure 6-1. A simple event-driven service that looks up the data it needs as it processes messages

So a single event stream is processed, and lookups that pull in any required data are performed inline. The solution suffers from two problems:

- The constant need to look things up, one message at a time.
- The payment and order are created at about the same time, so one might arrive before the other. This means that if the order arrives in the email service before the payment is available in the database, then we'd have to either block and poll until it becomes available or, worse, skip the email processing completely.

The Pure (Stateless) Streaming Approach

A streaming system comes at this problem from a slightly different angle. The streams are buffered until both events arrive, and can be joined together ([Figure 6-2](#)).

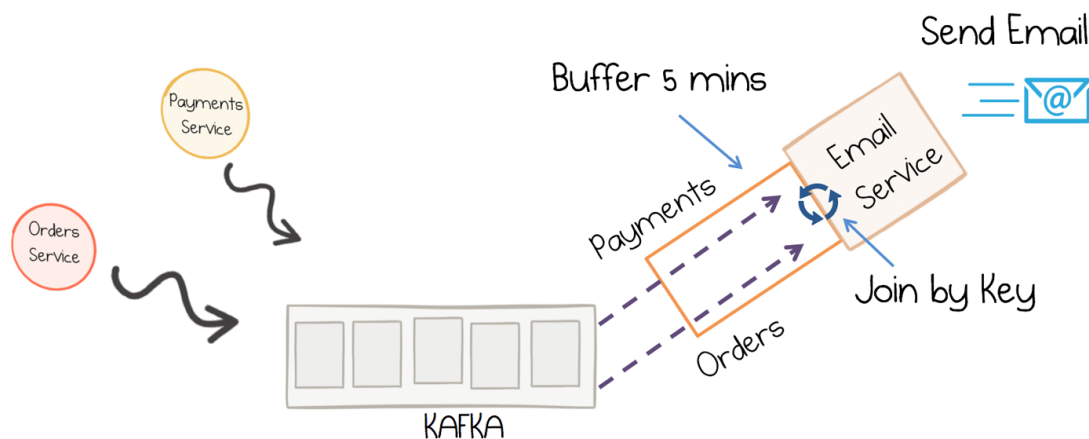


Figure 6-2. A stateless streaming service that joins two streams at runtime

This solves the two aforementioned issues with the event-driven approach. There are no remote lookups, addressing the first point. It also no longer matters what order events arrive in, addressing the second point.

The second point turns out to be particularly important. When you're working with asynchronous channels there is no easy way to ensure relative ordering across several of them. So even if we know that the order is always created before the payment, it may well be delayed, arriving the other way around.

Finally, note that this approach isn't, strictly speaking, stateless. The buffer actually makes the email service stateful, albeit just a little. When Kafka Streams restarts, before it does any processing, it will reload the contents of each buffer. This is important for achieving deterministic results. For example, the output of a join operation is dependent on the contents of the opposing buffer when a message arrives.

The Stateful Streaming Approach

Alas, the data flowing through the various event streams isn't always enough—sometimes you need lookups or enrichments. For example, the email service would need access to the customer's email address. There will be no recent event for this (unless you happened to be very lucky and

the customer just updated their details). So you'd have to look up the email address in the customer service via, say, a REST call ([Figure 6-3](#)).

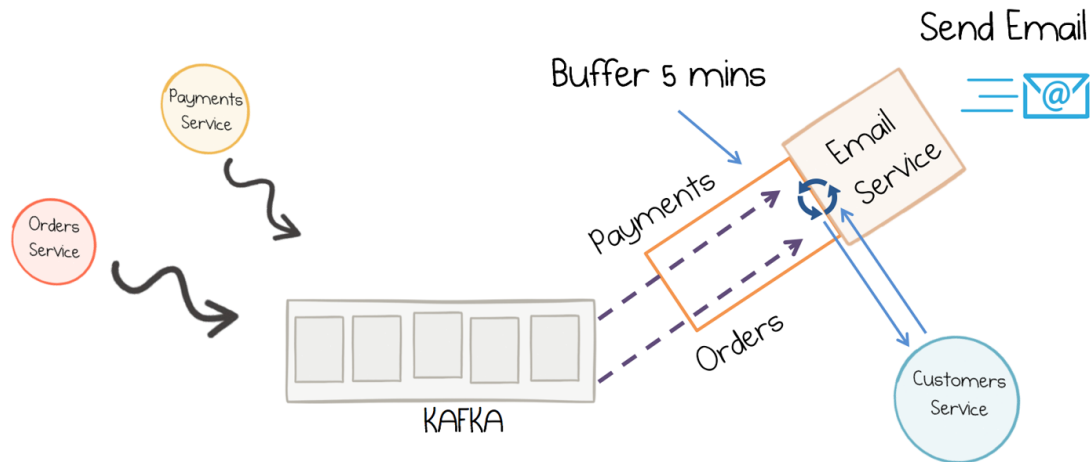


Figure 6-3. A stateless streaming service that looks up reference data in another service at runtime

This is of course a perfectly valid approach (in fact, many production systems do exactly this), but a stateful stream processing system can make a further optimization. It uses the same process of local buffering used to handle potential delays in the orders and payments topics, but instead of buffering for just a few minutes, it preloads the whole customer event stream from Kafka into the email service, where it can be used to look up historic values ([Figure 6-4](#)).

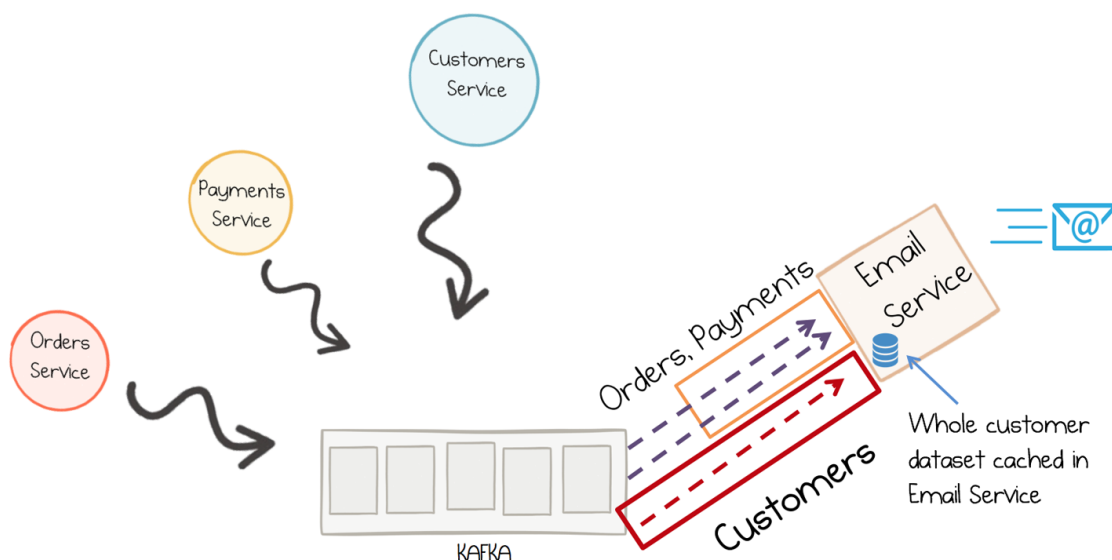


Figure 6-4. A stateful streaming service that replicates the Customers topic into a local table, held inside the Kafka Streams API

So now the email service is both buffering recent events, as well as creating a local lookup table. (The mechanics of this are discussed in more detail in [Chapter 14](#).)

This final, fully stateful approach comes with disadvantages:

- The service is now stateful, meaning for an instance of the email service to operate it needs the relevant customer data to be present. This means, in the worst case, loading the full dataset on startup.

as well as advantages:

- The service is no longer dependent on the worst-case performance or liveness of the customer service.
- The service can process events faster, as each operation is executed without making a network call.
- The service is free to perform more data-centric operations on the data it holds.

This final point is particularly important for the increasingly data-centric systems we build today. As an example, imagine we have a GUI that allows users to browse order, payment, and customer information in a scrollable grid. The grid lets the user scroll up and down through the items it displays.

In a traditional, stateless model, each row on the screen would require a call to all three services. This would be sluggish in practice, so caching would likely be added, along with some hand-crafted polling mechanism to keep the cache up to date.

But in the streaming approach, data is constantly pushed into the UI ([Figure 6-5](#)). So you might define a query for the data displayed in the grid, something like `select * from orders, payments, customers where...`. The API executes it over the incoming event streams, stores the result locally, and keeps it up to date. So streaming behaves a bit like a decoratively defined cache.

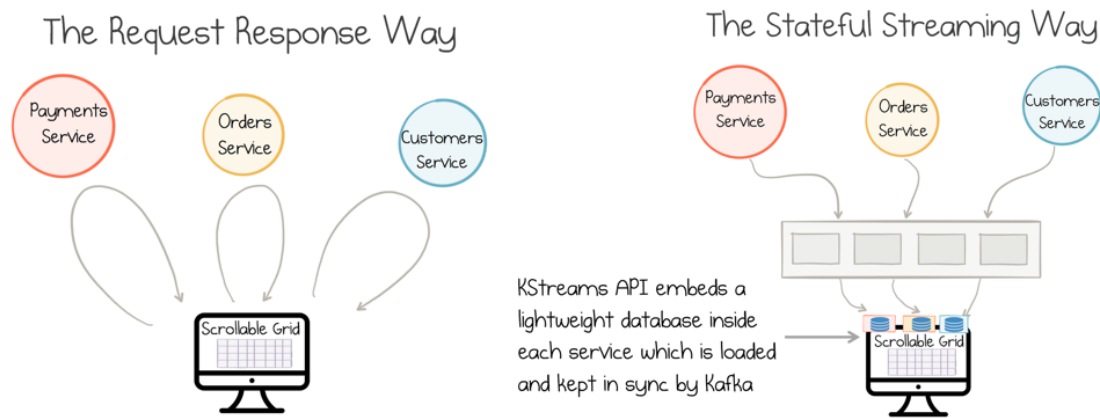


Figure 6-5. Stateful stream processing is used to materialize data inside a web server so that the UI can access it locally for better performance, in this case via a scrollable grid

The Practicalities of Being Stateful

Being stateful comes with some challenges: when a new node starts, it must load all stateful components (i.e., state stores) before it can start processing messages, and in the worst case this reload can take some time. To help with this problem, Kafka Streams provides three mechanisms for making being stateful a bit more practical:

- It uses a technique called [standby replicas](#), which ensure that for every table or state store on one node, there is a replica kept up to date on another. So, if any node fails, it will immediately fail over to its backup node without interrupting processing unduly.
- [Disk checkpoints](#) are created periodically so that, should a node fail and restart, it can load its previous checkpoint, then top up the few messages it missed when it was offline from the log.
- Finally, [compacted topics](#) are used to keep the dataset as small as possible. This acts to reduce the load time for a complete rebuild should one be necessary.

Kafka Streams uses intermediary topics, which can be reset and re-derived using the Streams Reset tool.¹

NOTE

An *event-driven application* uses a single input stream to drive its work. A *streaming application* blends one or more input streams into one or more output streams. A *stateful streaming application* also recasts streams to tables (used to do enrichments) and stores intermediary state in the log, so it internalizes all the data it needs.

Summary

This chapter covers three different ways of doing event-based processing: the simple event-driven approach, where you process a single event stream one message at a time; the streaming approach, which joins different event streams together; and finally, the stateful streaming approach, which turns streams into tables and stores data in the log.

So instead of pushing the state problem down a layer into a database, stateful stream processors, like Kafka's Streams API, are proudly stateful. They make data available wherever it is required. This increases performance and autonomy. No remote calls needed!

Of course, being stateful comes with its downsides, but it is optional, and real-world streaming systems blend together all three approaches. We go into the detail of how these streaming operations work in [Chapter 14](#).

¹ See <http://bit.ly/2GaCRZO> and <http://bit.ly/2IUPHJa>.

[Support](#) [Sign Out](#)

©2022 O'REILLY MEDIA, INC. [TERMS OF SERVICE](#) [PRIVACY POLICY](#)